

ДОПОЛНЕНИЕ · КОМПАНЬОН К «ИНЖЕНЕРНОЙ БАЗЕ»

Архитектура, читаемость, анти-раздутие

Три конкретных вопроса из практики Nila AI: какие Claude Code skills использовать для архитектуры, как писать код, читаемый человеком, и — главное в этот раз — как не допустить раздутия функций и файлов до 500+ строк с *первого коммита*, а не разбирать это постфактум.

- 01 Skills для архитектуры
- 02 Читаемость
- 03 Анти-раздутие

01

Skills для архитектуры кода

Пять skills в системе покрывают разные моменты цикла разработки — вопрос не «какой один использовать», а «что запускать на каком этапе».

SKILL	КОГДА	ЧТО ДЕЛАЕТ
improve-codebase-architecture	Раз в N спринтов, или когда что-то ощущается архитектурно «не так» — не на каждый PR	Диагностика по Ousterhout: ищет shallow modules (интерфейс ≈ реализации, ничего не скрывает) и предлагает deepening. Тест: удаление модуля концентрирует сложность → модуль оправдан; сложность расплзается по вызывающим → модуль shallow. 3 фазы: Explore → HTML-отчёт с кандидатами → обсуждение конкретного кандидата.
simplify	Сразу после фичи, до code-review	Чистит reuse/simplification/efficiency в изменённом коде, пока свежо в контексте. Багов не ищет — только качество.
code-review	Перед мержем, на весь diff	Correctness-баги + reuse/simplification findings. Effort low/medium — быстрая проверка, high/max — для рискованных изменений.
verify	После нетривиального изменения, перед коммитом	Гоняет фичу end-to-end по-настоящему — ловит «тесты зелёные, фича не работает».
security-review	Перед деплоем изменений auth/	Отдельный проход по security.

ПОРЯДОК НА ФИЧУ

написал код → simplify → code-review → verify → коммит. improve-codebase-architecture — отдельный ритм, не привязан к конкретному PR.

02

Читаемость кода для человека

SPEC-блоки, intent-naming, word boundary и why-комментарии из основного свода — это уровень строки и функции. Здесь — уровень модуля, которого не хватало.

Deep module principle (Ousterhout)

Модуль тем лучше, чем меньше его интерфейс относительно того, что он скрывает внутри. Признак плохого модуля: интерфейс почти дублирует реализацию — тонкие get/set-обёртки без логики. Такие функции разумнее заинлайнить, а не выносить «ради вынесения».

Один уровень абстракции на функцию

Внутри функции не смешивать «что делать» (`validateUser(); chargeCard(); sendReceipt();`) и «как именно» (конкретный SQL, парсинг строки). Смешение уровней внутри одной функции — сигнал extract.

Guard clauses вместо вложенности

Ранний `return / throw` на невалидный случай в начале функции — вместо оборачивания всего тела в `if (valid) { ... }`. Каждый уровень вложенности — нагрузка на читателя.

Именованные булевы вместо сложных условий

```
// вместо inline-выражения на 3 условия:  
if (isEligibleForTrial && !hasActiveSubscription && daysSinceSignup < 14) { ... }  
  
// вынести в говорящее имя:  
const canStartTrial = isEligibleForTrial && !hasActiveSubscription && daysSinceSignup < 14;  
if (canStartTrial) { ... }
```

Единый словарь терминов (domain glossary)

Если `knows` в коде используется то `goal`, то `objective`, то `target` — это создаёт

Если «цель» в коде называется `to do`, `to observe`, `to target` — это создает ложные концептуальные различия. Завести `CONTEXT.md` / `LANGUAGE.md` с первого дня проекта (тот же паттерн использует `improve-codebase-architecture`) и вписывать туда новые термины по мере кристаллизации.

Тесты как документация интерфейса

Если объяснить, что делает функция, можно только прочитав её тело, а не сигнатуру и тесты — интерфейс недостаточно ясен.

03

Анти-раздутие — проактивно, с первого коммита

В прошлый раз лимиты появились уже при разборе файла на 6000+ строк. Задача теперь — не дать раздуться физически, настроив пороги в скелете проекта до первой строчки бизнес-логики.

- 1
- Жёсткие правила ESLint — `error`, не `warn`.**

не останавливает — его научаются игнорировать за неделю.

`error` ломает `npm run lint`, а значит ломает CI.

```
rules: {
  'max-lines-per-
  'max-lines': ['
  'complexity': [
  'max-depth': ['
  'max-params': [
  'sonarjs/cognit
}
```

- 2
- Pre-commit hook (husky + lint-staged)**

— превышение лимита не попадает в git вообще, вместо того чтобы ловиться на code review постфактум.

```
npx husky add .husky/p
// lint-staged: eslint
```

- 3
- Тот же линт — обязательный шаг CI**

, не только локальный hook. Hook можно обойти `--no-verify`, CI — нельзя.

- 4
- Архитектурный скелет с первого дня**

, не после 6000 строк — сразу структура `use-case-per-file / business-component` по доменам (`entry-points/ domain/ data-access/`). Если с первого дня нет единого файла, куда «удобно дописать ещё один `if`» — раздувать физически некуда.

- 5
- Триггер `extract` по сути, не только по**

как только функция начинает делать 4-е по счёту дело (валидация → бизнес-правило → запись в БД → форматирование ответа — уже 4) — остановиться и вынести, не дожидаясь порога линтера. Линтер — строчка не опий, если триггер пропустим, а не основной

только по страховка на случай, если триггер пропустили, а не основной
строкам: механизм.

- 6 **Периодический** , не @genese/complexity или escomplex по всей кодовой
отчёт по всему только базе. Линтер на
проекту по diff: PR ловит новые
 раз в нарушения, но
 спринт не покажет, что
 — файл вырос до
 280 строк за 15
 маленьких
 «безопасных»
 PR подряд.

ПОЧЕМУ ВАЖНО ERROR, А НЕ WARN

Прошлый монолит на 6000+ строк начинался как обычный файл — рос через десятки отдельных «безопасных» изменений, ни одно из которых само по себе не выглядело как раздутие. warn -порог не останавливает это накопление; останавливает только то, что физически ломает коммит или сборку.

ai_agent_chat / Nila AI → перенос практик в новый wellness-чат

